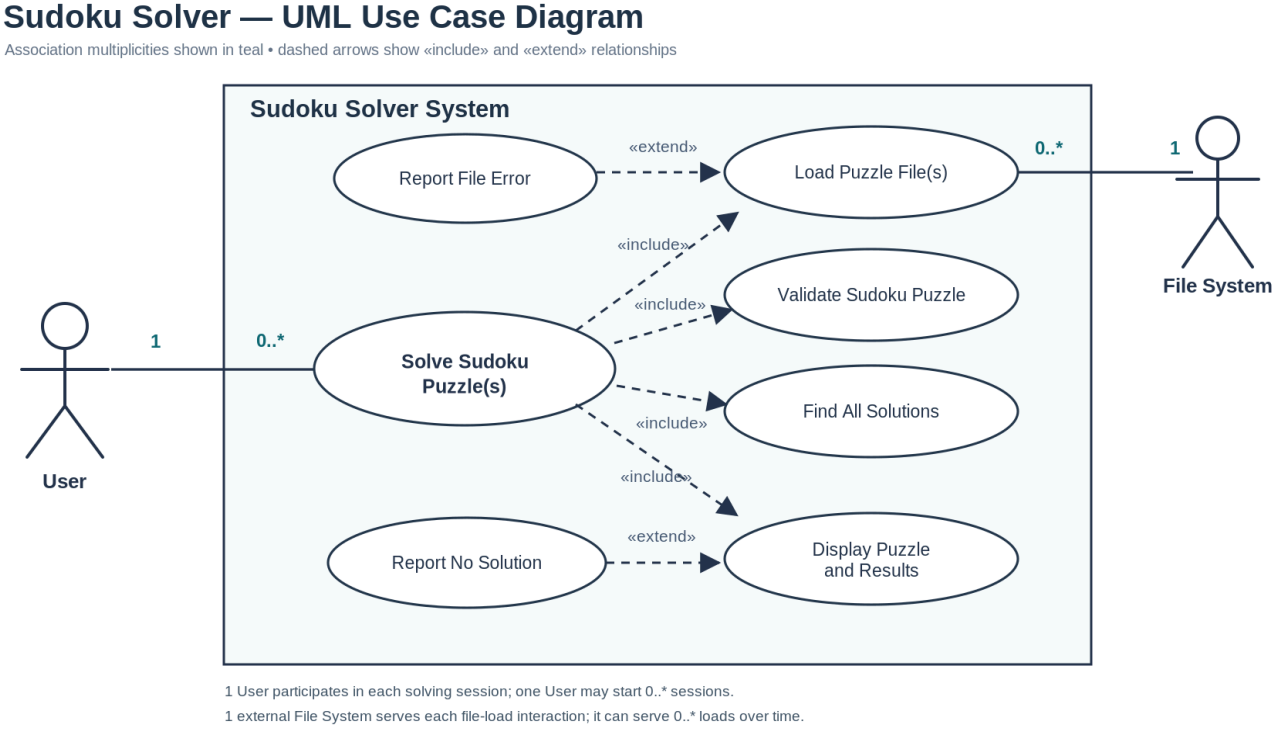


a. Identify/discover a use case: The use case is to solve Sudoku puzzle(s). The primary actor in the use case is the user. The purpose is to allow the user to submit one or more Sudoku puzzles and return all possible solutions to the user.

b. Provide a brief description for the use case: The use case allows the user to solve Sudoku puzzles by reading puzzle data from input text files. The program uses recursive backtracking to find all possible solutions to the given user puzzle. The program then prints all solutions back to the user. If there is no solution, the system indicates that as well.

c. Provide a step-by-step flow (outline) of the events for the use case:

1) The user starts the Sudoku solver program.
2) The system recognizes puzzle files indicated in the program.
3) The system opens and reads puzzle files.
4) The system validates puzzle format and sizing.
    a) If the sizing or format is bad, the program skips the given file.
    b) If the sizing matches expected input, the program continues.
5) The program ensures that starting numbers follow Sudoku rules.
    a) If the numbers on the puzzle do not follow Sudoku rules, the program rejects the puzzle.
    b) If the numbers do follow Sudoku rules, the program continues.
6) The system identifies an empty cell and determines its possible values.
7) The system uses recursive backtracking to continue to find possible values and solutions to the puzzle.

8) The system prints solutions as they are found.
9) The system repeats the process for remaining puzzle files and prints solution results.
10) The program terminates when all puzzle files have been processed.

d. Provide a detailed description for the use case:

1) The user starts the Sudoku solver program.

The user runs the program. They can either provide the name of a specific puzzle file or use the five default puzzle files.

2) The system recognizes puzzle files indicated in the program.

The system opens the requested files, either from the user or the default files.

3) The system opens and reads puzzle files.

The system will identify underscores as empty cells and digits 1-9 as filled cells.

4) The system validates puzzle format and sizing.

The system checks that the puzzle has 81 cells formatted into a 9 by 9 grid.

a) If the sizing or format is bad, the program rejects the file.

The program will print "Unable to read a valid puzzle file." and move on to the next puzzle file.

b) If the sizing matches the expected input, the program continues.

5) The program ensures that starting numbers follow Sudoku rules.

The system checks that each row, column, and 3x3 grid contains no duplicate number values.

a) If the numbers on the puzzle do not follow Sudoku rules, the program considers the puzzle to have no valid solution.
b) If the numbers do follow Sudoku rules, the program continues.

6) The system identifies an empty cell and determines its possible values.

The system searches the Sudoku board for empty cells. It determines which digits 1-9 can be in each cell according to Sudoku rules. If a cell has only a single candidate, that cell is immediately chosen.

7) The system uses recursive backtracking to continue to find possible values and solutions to the puzzle.

The system places a possible number in the selected empty cell. It recursively repeats to fill additional empty cells. If no valid digit can be placed, the system backtracks to the previously selected cell. The “bad” number is removed, and another candidate is chosen. This recursive process is continued until all solutions are found.

8) The system prints solutions as they are found.

When a completed solution is recognized, the system prints the completed 9x9 Sudoku board. The solution counter is increased by 1. If no solution is found, the program returns “no solution found” for the specific puzzle file.

9) The system repeats the process for remaining puzzle files and prints solution results.
The system repeats the last steps for each additional puzzle file given. At this point, the program prints the solution results. This includes the number of solutions or “no solutions found”.

10) The program terminates when all puzzle files have been processed.
The program closes after all solutions have been displayed to the user.